

```
CREDITCOIN CC3 · ATTESTCOIN NATIVE QUERY VERIFIER  
precompile 0x00000000000000000000000000000000FD2
```

DORAHACKS · DEFI TRACK

Proves that transaction A
executed **before** transaction B
inside an Ethereum block — a
fact carried in no payload,
readable by no oracle.

CC3 testnet

index41

All you had was a screenshot.

You paid a relay so you
wouldn't be sandwiched.
You were sandwiched
anyway.

You complained. The relay denied it. That was the
end of it — because **a screenshot is not evidence a
contract can read.**

WHAT THE VICTIM ACTUALLY HAS		3 HASHES · 1 BLOCK
POSITION	TRANSACTION	CLAIM
..	0xec3777f9d0e5...79af618436 someone bought first — allegedly	unproven
..	0x7b054188f739...cbb44485a0 your swap, at the worse price	unproven
..	0xb0cae362c6a6...10b65be23a someone sold after — allegedly	unproven
An unordered set. The harm is entirely about order, and the order is the part nobody can show.		

No payload carries its position.

THE VICTIM’S TRANSACTION, IN FULL

ETH_GETTRANSACTIONBYHASH + RECEIPT

0x7b054188f739937a2fc2c9257b48f2ccab0ec4440ce5eda7f2ec5ccbb44485a0

from 0x51f400b9770ad2bddb7cf74664f5cd1daf6a1410

block 25764741 · 240 transactions

nonce · gas · value · input · logs · status all present

index — does not exist —

not in the payload

not in the receipt

not in any log

not returned by eth_call

A searcher bought in front of that swap, let it execute into the worse price, and sold behind it. **The harm is entirely a fact about ordering** — and a transaction does not carry its own position.

THE TRAP

A block explorer will **show** you a position — it indexed the block off-chain, on its own authority. It cannot **prove** one to a contract.

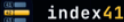
A sandwich is three transactions that only become an attack once you can show that one landed **between** the other two. “Between” is exactly what no payload contains, so the victim’s case dies at the point where it would have to become evidence.

A bonded court for ordering.

A relay bonds CTC behind its no-sandwich promise. A victim submits three same-block hashes. **One Creditcoin transaction verifies them, orders them, and pays out of the bond.**

- 01** `postBondFor · declareCoverage`
the relay stakes, and names what it covers
- 02** `verifyAndEmit ×3`
three legs, one shared continuity proof
- 03** `calculateTxIndex ×3`
position out of merkle laterality — a free view
- 04** `require(front < victim < back)`
plus same pool, shared sender, priority-fee order
- 05** `bond → victim`
harm = the attacker’s realized profit, from proven logs

index41 — npm install && npm run dev → localhost:3000

index41

THE PROOFMECHANISMPROVENANCEEVIDENCEFAQFOR JUDGES

1

live on CC3 testnet

CREDITCOIN CC3 · LIVE

ATTESTCOIN NATIVE QUERY VERIFIER

DEFI

Position inside a block was a claim. Now it is a fact.

index41 proves that transaction A executed before transaction B inside an Ethereum block — by reading each transaction’s ordinal position out of the left/right laterality of its merkle authentication path. A relay bonds CTC behind “you will not be sandwiched”. When it happens, the contract asserts `front < victim < back`, computes the attacker’s realized profit from proven logs, and pays the victim from the bond.

3

the assertion, in one line

WATCH THREE ROWS LIGHT UP ↓

CONTRACT ON BLOCKSCOUT ↗

2

decoded on every page load, not typed in

14 · 15 · 16

positions recovered on-chain from the precompile’s own logs

1.456%

of MAX_GAS_CAP — 1,092,100 gas

three verifications, one transaction

8

120

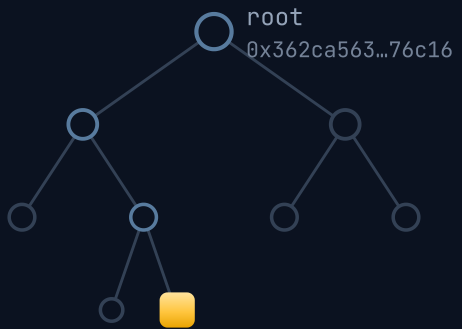
index41

04 / 10

Laterality is the index.

1 · THE BLOCK

Ethereum mainnet
#25764741
240 transactions · merkle depth 8



the gold leaf is our transaction

EACH LEAF IS `abiEncode(tx, rx)`

A block commits its transactions as a tree. Walking leaf to root, every sibling sits on one side.

Those sides are the position, written in binary.

Attestcoin proves transaction HISTORY, never state.

2 · THE AUTHENTICATION PATH

DEPTH	SIBLING	SIDE	BIT	BIT · 2^d
0	0x3d20aad9...c51a2	R	0	+0
1	0x39a67f4c...6f4d4	L	1	+2
2	0x022956d4...ccb91	L	1	+4
3	0xe81eba4d...78e7b	L	1	+8
4	0xb168d936...d9605	R	0	+0
5	0xefc8dd4f...ba978	R	0	+0
6	0x7af37fa7...583c0	R	0	+0
7	0xef11896d...16518	R	0	+0

L = SIBLING ON THE LEFT, SO THIS NODE WAS THE RIGHT CHILD, SO THE BIT IS 1

PATH

R L L R R R R

= INDEX

2 + 4 + 8

14

Read leaf→root with R = 0 and L = 1, the path IS the index.

3 · ON CREDITCOIN, ON CHAIN

INativeQueryVerifier

precompile 0x...0FD2 · exactly two functions

verifyAndEmit(...) ×3, one tx

calculateTxIndex(...) view · 0 gas

Index41.proveSandwich

require(front < victim < back)

same pool · shared sender

front-run outbid the victim

harm = realized profit

Swap logs, decoded by EvmV1Decoder

– the same bytes the precompile verified

bond → victim

processedQueries[qid] = true – replay sealed

Reads only. Creditcoin cannot write back to Ethereum, and does not need to.

Three rows. One block. Proven.

240 TXS · MERKLE DEPTH 8

RULED ON CREDITCOIN CC3

1 index — emitted by the precompile itself

2 8 siblings, 8 bits — the position in binary

POSITION	TRANSACTION	PATH	LEAF → ROOT	ROLE
	... 14 transactions above — index41 never touches them			
14	0xec3777f9d0e55d03b9caa3a4b8a786dd62e16eeb327a9f1c45dfbc79af618436 the searcher moves the pool first	R L L L R R R R		searcher buy front-run
15	0x7b054188f739937a2fc2c9257b48f2ccab0ec4440ce5eda7f2ec5ccbb44485a0 the swap that gets the worse price	L L L L R R R R		the victim victim
16	0xb0cae362c6a6dcf08f4adfc1d510cdda851271a913cdea4bc81da010b65be23a the searcher exits into the damage	R R R R L R R R		searcher sell back-run
	... 223 transactions below — index41 never touches them			
14 < 15 < 16 FRONT-RUN BEFORE VICTIM BEFORE BACK-RUN ✓ 3 one require(), inside contract control flow — it reverts if it does not hold				

THE RULING — ALL FIVE LOGS OF ONE CREDITCOIN TRANSACTION	STATUS 1 · BLOCK 5,317,821 · 1,092,100 GAS
log 0 TransactionVerified(chainKey=3, height=25764741, txIndex=14) ← written by the precompile, not by us	
log 1 TransactionVerified(chainKey=3, height=25764741, txIndex=15)	
log 2 TransactionVerified(chainKey=3, height=25764741, txIndex=16)	
log 3 SandwichProven(0x111112...373911, 25764741, 14, 15, 16, harm=219708, paid=219708)	
log 4 HarmPaid(victim=0x51f400...6a1410, relay=0x9436ed...c45e24, 219708) ← the attacker's own realized profit	

VERIFY IT creditcoin-testnet.blockscout.com/tx/0xd136dea0524b7e0e9eba54bf9724eec78597c2598047a96849af727f4d243810

Depth, stated as arithmetic.

30 surfaces doing real work on a clean default run — `npm run prove`, zero flags

36 load-bearing across the default and `--kill-hosted` runs

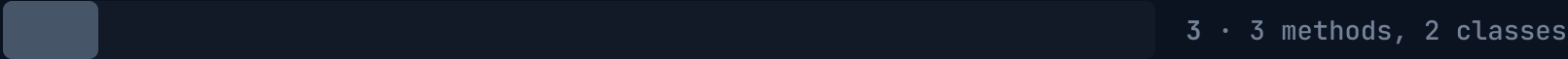
24 of those 36 are **undocumented** — absent from docs.creditcoin.org

3 the entire surface the four official examples exercise: **3 methods on 2 classes**. That is the field’s baseline.

The number to hold us to is **30**, not 36 — and the gap is spelled out, because a surface count is exactly the kind of number people use to smuggle a claim past a reader.

SURFACES MADE LOAD-BEARING OF 325 CATALOGUED • 265 OF THEM UNDOCUMENTED

THE FOUR OFFICIAL EXAMPLES



INDEX41 • CLEAN DEFAULT RUN



WHERE THE 36 SIT

8 + 4 + 9 + 3 + 5 + 4 + 3

chainInfo 8 • blockProver 4 • proofProvider 9 • encoding 3
utils 5 • proof-gen API 4 • on-chain 3

`INativeQueryVerifier.calculateTxIndex`

Undocumented. Untouched by all four official examples. **The product.** Delete any one of the 36 and something stops being checkable.

THE BASELINE, NAMED

EXAMPLES_REPO • EXACTLY ONE FILE IMPORTS THE SDK

`ProofBuilder ctor` → `waitUntilHeightAttested` → `getProof`
plus `getLatestAttestedHeightAndHash`

That list is the whole of it. Every competitor starts from the same four lines.

Remove it and nothing remains.

INSTEAD OF ATTESTCOIN	WHAT YOU WOULD HAVE TO BUILD	WHAT THE CONTRACT WOULD THEN BE TRUSTING
An ordering oracle	It does not exist. No production network sells “transaction Oxec37... was the 14th of block 25,764,741” — not a price, not a feed, an arbitrary historical fact.	—
A signer committee	Choose signers, bond them, define slashing, and run it for the lifetime of every bond you underwrite.	An ECDSA signature over a claim . Only one of the two designs survives the committee being wrong.
A bespoke indexer	Ordering alone is not a sandwich: you also need the same pool, the shared sender, the priority-fee order and the Swap amounts — each a separate assertion by a separate service.	Several correlated claims instead of one proof . A new place to lie per field.
A message bridge	Its own validator set, to carry any of this from Ethereum to Creditcoin at all.	An honest-majority assumption , under a contract whose selling point is taking nobody’s word.

index41 today	calculateTxIndex over the block’s own merkle commitment, plus EvmV1Decoder over the same verified bytes.	Nothing. It re-derives the fact and reverts if it does not hold.
---------------	--	--

DEPTH REFUSED WHERE IT WOULD BE DECORATION

- ☐ queryBuilder — 62 surfaces, skipped. EvmV1Decoder already returns those fields from the same verified bytes.
- ☐ proofProvider.mergeProofs — throws on non-contiguous ranges, and three legs of one block are not a range.
- ☐ verifyBatch / verifyAndEmitBatch — TypeScript-side only. There is no on-chain batch verify, which is exactly why a ruling makes three sequential calls.

A surface count that includes decoration is a worse number than a smaller one that does not. **Every one of the 36 (30 doing real work on a clean default run) is named, with what it is load-bearing for.**

Measured, then disclosed.

●●● npm run spike - docs/spike-output.txt, committed

```
— 7. THE GAS GATE - 3x verifyAndEmit in ONE Creditcoin transaction —
eth_call      proveOrder → [14, 15, 16] (no gas spent)
CC3 tx hash   0x42f725dd8c875185b216b6bbda01d37e12f605c4e8dbd832e7649a8abeeedd02
block        5317584   status 1
GAS USED      292376
MAX_GAS_CAP   75000000
precompile    TransactionVerified(chainKey=3, height=25764741, txIndex=14)
precompile    TransactionVerified(chainKey=3, height=25764741, txIndex=15)
precompile    TransactionVerified(chainKey=3, height=25764741, txIndex=16)
               the precompile ITSELF reports 14, 15, 16 -
               not our contract, not our script.
```

292,376 ÷ 75,000,000

0.390% of MAX_GAS_CAP

The whole project lived or died on whether three `verifyAndEmit` calls fit in one transaction. It was made a day-3 go/no-go gate and spiked against the live network before a line of product code. The full ruling, with decoding, harm and payout, costs **1,092,100 gas** — **1.456% of the cap**.

SHIPPED, AND CHECKABLE BY A STRANGER

- **120 Foundry unit tests**, 4 suites, 0 failed — including an exhaustive round-trip over **all 256 leaves** of the depth-8 tree
- **48 Playwright runs**, chromium + Pixel 7, and none of them assert 14, 15 or 16 — they assert invariants
- Contract **deployed and source-verified** on CC3, 15,213 bytes, and it has ruled once
- **219,708 wei** paid from the bond to the address the proof named — paid == computed
- The same ruling reproduces with the hosted prover switched off entirely (`--kill-hosted`)

DISCLOSED, NOT DISCOVERED

- ☐ **It cannot prove state.** Attestcoin commits transaction history, so harm is the attacker’s realized profit — never a counterfactual.
- ☐ **No on-chain batch verify.** A ruling is three sequential calls in one transaction.
- ☐ **3 of the 120 unit tests prove the mock**, not the precompile — and are named so. The real precompile was confirmed live on CC3.
- ☐ **Testnet, unaudited, one relay, one ruling.** The bond is play money until it is not.

Score the depth. Click the hash.

- 01 **Open the ruling on Blockscout.** The contract is source-verified, so the explorer decodes the events itself — you will read 14, 15, 16 in the precompile’s own logs, not in ours.
- 02 **Run it without a key.** `npm install && npm run dev` — no .env, no wallet prompt, real data on the default path.
- 03 **Then score the one published criterion.** Depth of Attestcoin utilization: 30 surfaces doing real work on a clean default run, 24 of the 36 undocumented, against a field baseline of 3.



SCAN → THE RULING, ON CHAIN
creditcoin-testnet.blockscout.com
/tx/0xd136dea052...7f4d243810

status 1 · block 5,317,821
1,092,100 gas · 5 logs

LIVE AND VERIFIABLE RIGHT NOW

The ruling	blockscout · tx 0xd136dea052...7f4d243810
The contract	0xb37Bc52b9d6f7431Ba8Be4deD4f53281Efb10eC2
The block	eth.blockscout.com/block/25764741
The event	dorahacks.io/hackathon/buidl-ctc-2026-fall

A transaction’s position is the one fact no payload carries. Creditcoin can read it — and the bond has already paid.



index41

Creditcoin CC3 testnet · chainId 102031